

PRÁCTICA: LLMs aplicados a Finanzas

Un agente investigador sobre informes 10-K de la SEC

Datos básicos:

- Grupos de 3 estudiantes.
- Entrega el día 23 de septiembre a través del aula virtual, a las 23:59.
- El entregable es un repositorio de GitHub y un informe en formato PDF.
- El día 24 hará una presentación cada grupo de 8 minutos más preguntas.
- El día 24, al empezar la clase, se entregan 10 preguntas ciegas que no ha visto nadie. Se ejecutan en el aula, contra el repositorio ya entregado, y el resultado entra en la presentación.

Requisitos:

- API Key de algún servicio de IA que será el que proporcione el “cerebro” del agente. Las más populares son OpenAI, Google o Anthropic, pero existen alternativas atractivas como OpenRouter. Se podrán utilizar modelos en local, pero su capacidad deja que desear salvo para hardware potente.
- Cuenta de Google con acceso a Google Colab o un entorno local funcional equivalente.
- Cuenta de Github para almacenar el código.

1. Objetivo de la práctica

Construir y evaluar un agente que responda preguntas sobre informes anuales 10-K presentados ante la SEC, citando de dónde sale cada dato. El agente dispone de cuatro herramientas con coste y precisión muy distintos, y la tarea central es que decida bien entre ellas: una consulta numérica exacta y gratuita contra los datos XBRL, una búsqueda difusa sobre el texto, o la lectura completa de una sección de cuarenta páginas. Se evaluará el sistema contra un conjunto de preguntas con respuesta conocida, midiendo calidad, coste y latencia.

La idea que gobierna la práctica es que el retrieval no es la arquitectura, sino una herramienta más. De ahí se sigue el criterio de corrección: un agente que acierta el revenue de NVIDIA leyéndolo de la prosa de una tabla partida está mal aunque el número sea correcto, porque ese camino no generaliza. Acertar por el camino equivocado cuenta como fallo, y hay un evaluador escrito para detectarlo.

El texto de esta práctica es extenso y puede parecer confuso inicialmente, pero todo se verá en clase.

2. Contexto del problema

La práctica se desarrolla en forma de taller semi-guiado, estructurado en dos sesiones de 2,5 horas cada una:

- **Herramientas y bucle del agente** (Sesión 1, 10 de septiembre): anatomía de un 10-K y su coste en tokens, escritura de las herramientas, bucle de razonamiento y acción escrito a mano, y el mismo agente con framework.
 - Trabajo autónomo post clase: baseline corriendo y las 20 preguntas propias escritas. Sin las dos cosas, la sesión 2 no rinde: empieza ejecutando vuestro baseline y clasificando sus fallos.
- **Robustez, guardrails y evaluación** (Sesión 2, 17 de septiembre): qué había dentro de la caja negra de la búsqueda, mejora medida del retrieval, estado y preguntas comparativas, middleware de control y los tres evaluadores.
- **Exposición de las soluciones** (Sesión 3, 24 de septiembre): ejecución de las 10 preguntas ciegas y presentación y defensa técnica de los resultados, 8 minutos más preguntas.

La tarea se entregará justo antes de la sesión 3.

3. Datos disponibles

Se utilizará un corpus de informes 10-K descargados de SEC EDGAR: seis grandes tecnológicas estadounidenses (NVDA, MSFT, AAPL, GOOGL, META y AMZN) por dos ejercicios fiscales (FY2024 y FY2025), con los apartados 1A (factores de riesgo), 7 (discusión de la dirección), 7A (riesgo de mercado) y 8 (estados financieros). Son 48 secciones y 650.000 tokens totales aproximadamente.

El corpus se entrega ya construido, en dos ficheros ZIP de 5,6 MB entre los dos:

- corpus_miax_2026.zip
- indice_faiss.zip

No hay que descargar nada de EDGAR: la SEC limita a 10 peticiones por segundo y bloquearía a toda la clase a la vez. Se proporciona el notebook de la sesión 1 con la celda de instalación, la de montaje del corpus y las herramientas de partida.

El corpus consta de estos ficheros:

- **secciones.jsonl:** el texto íntegro de cada sección, una línea por empresa, ejercicio y apartado. Es lo que sirve la herramienta read_section.
- **chunks.jsonl:** ese mismo texto troceado en 1.749 fragmentos de unos 500 tokens con solapamiento de 80, cada uno con su chunk_id, que es lo que permite verificar una cita. Es lo que sirve search_filings.
- **xbtrl_facts.parquet:** 135 hechos numéricos tal y como los reportó cada compañía. Es la fuente autorizada para cualquier cifra, y el ground truth de la evaluación.
- **indice/:** un índice FAISS ya construido sobre los 1.749 fragmentos con el modelo de embeddings BAAI/bge-small-en-v1.5. Su manifiesto documenta el prefijo que hay que poner en la consulta y no en los fragmentos.

Tres avisos sobre estos datos, que ahorran tiempo:

- **fiscal_year no es el año de presentación:** los seis emisores cierran ejercicio en cuatro meses distintos, elegidos así a propósito. El FY2025 de NVIDIA cerró en enero de 2025 y el de Alphabet en diciembre de 2025, presentado en 2026. Hay que fiarse del campo, no de la fecha.
- **El concepto XBRL del ingreso no es universal:** NVIDIA usa Revenues; Apple, Microsoft, Meta y Amazon usan RevenueFromContractWithCustomerExcludingAssessedTax; y Alphabet etiqueta los dos en FY2024 pero solo Revenues en FY2025. Hay que mirar el fichero, nunca razonar por analogía con otra compañía.
- **Hay huecos reales, y son preguntas legítimas:** Amazon no reporta GrossProfit, Liabilities ni ResearchAndDevelopmentExpense en us-gaap, y Meta y Alphabet tampoco GrossProfit. Preguntar por el margen bruto de Amazon tiene como respuesta correcta que no está en el corpus. Que el agente lo diga en vez de inventarse una cifra es relevante.

4. Tarea del estudiante

El estudiante tiene cinco tareas principales:

1. **Cinturón de herramientas:**

Reimplementar o mejorar las herramientas proporcionadas en clase:

```
list_available()
get_xbrl_fact(ticker, fiscal_year, concept)
search_filings(query, ticker, fiscal_year, item, k)
read_section(ticker, fiscal_year, item)
```

Los nombres y los parámetros son contrato y no se cambian: las preguntas ciegas del día 24 se ejecutan contra ellos. El docstring de cada herramienta es lo único que ve el modelo para decidir si la llama, así que escribirlo bien es parte de la tarea y no documentación.

2. **El agente y sus guardrails:** montar el agente con salida estructurada obligatoria, con los campos respuesta, cifra, unidad, ticker, ejercicio, fuente, cita y chunk_id. Añadir un límite de llamadas a herramienta por invocación y un middleware propio que extraiga las cifras de la respuesta y las contraste contra los datos XBRL, devolviendo el desajuste al modelo cuando no cuadren.
3. **Golden set propio:** escribir 20 preguntas con respuesta conocida en formato JSONL, de las cuales al menos 6 deben ser comparativas entre dos ejercicios. Las otras dos familias son extractiva (mide retrieval y trazabilidad) y numérica (mide el guardrail contra XBRL).
En las extractivas la verdad se ancla a una frase literal del informe, de una sola frase, y no a un chunk_id: el identificador cambia en cuanto se re-trocea el corpus, y el grupo que mejore el troceado no puede salir penalizado por haberlo mejorado.
4. **Mejora medida del retrieval:** partiendo de la búsqueda densa que se entrega, aplicar y medir al menos el filtro por metadatos, la combinación de BM25 con búsqueda densa y la reescritura de la consulta con el propio modelo. Hay que

medir recall@k después de cada arreglo, contra el ancla de texto.

5. **Evaluación automática:** escribir tres evaluadores —que la cita exista y respalde de verdad lo que afirma, que la cifra coincida con XBRL dentro de una tolerancia documentada, y que la trayectoria haya pasado por la herramienta que tocaba— y exponer dos funciones, responder(pregunta) y evaluar(ruta_jsonl), para poder ejecutar las 10 preguntas ciegas el día 24 sin tocar código.

5. Entregables

Se deberá entregar un github con el agente, el golden set y los resultados, y un informe en pdf a través del aula virtual.

Contenido obligatorio del GitHub:

- El código del agente, las cuatro herramientas y los tres evaluadores. El código debe generar todas las tablas reportadas.
- El golden set propio en JSONL, con 20 preguntas y al menos 6 comparativas, y que pase el validador que se entrega.
- Las funciones responder() y evaluar(), ejecutables sobre un clon limpio del repositorio sin editar nada. Conviene ensayarlo antes del día 23.
- Una tabla que compare el sistema baseline con el sistema final sobre el golden set propio, con aciertos por familia, recall@k del retrieval, coste medio por pregunta, latencia media y llamadas a herramienta por pregunta. Remarcar el mejor valor. El coste y la latencia son columnas de la tabla, no una nota al pie.
- Los ficheros de resultados del baseline y del sistema final, regenerables ejecutando el repositorio. Hay que guardar el baseline etiquetado antes de empezar a mejorarlo: es media tabla del informe.
- Ninguna clave de API en el repositorio.

Contenido de la presentación:

- La tabla baseline frente a final y su lectura: qué mejoró, cuánto, y a qué coste. La pregunta que se hará a todos los grupos es si las mejoras introducidas mejoraron algo de verdad y qué costaron.
- El resultado de las 10 preguntas ciegas, con su delta contra el golden set propio. Si baja, hay que explicar qué parte de la mejora era general y qué parte era memoria del conjunto con el que se iteró; eso es un hallazgo, no un suspenso.
- Cómo enruta el agente entre la herramienta exacta y la difusa, y qué guardrail lo protege de afirmar una cifra que no ha verificado.
- Qué se probó que no funcionó. Un arreglo razonable que no movió la métrica es un resultado.

6. Criterios de evaluación

- (30 %) Github
- (70 %) Presentación. Se presentará el pdf entregado en el aula virtual. Cada grupo hará una presentación de 8 minutos explicando sus resultados.

7. Anexo de código:

Las cuatro herramientas:

Las firmas son contrato. Podéis reimplementar el cuerpo entero, añadir parámetros con valor por defecto y añadir herramientas nuevas, pero no cambiar los nombres ni los parámetros existentes: el hold-out del día 24 se ejecuta contra vuestro agente y el evaluador de trayectoria busca estos nombres.

```
```python
@tool
def list_available() -> str:
 """Devuelve las empresas y ejercicios fiscales disponibles en el
 corpus."""

@tool
def get_xbrl_fact(ticker: str, fiscal_year: int, concept: str) -> str:
 """Devuelve el valor EXACTO de una magnitud financiera tal y como la
 compañía la reportó en XBRL. Es la fuente autorizada para cualquier
 cifra. Úsala SIEMPRE en lugar de leer un número del texto."""

@tool
def search_filings(query: str, ticker: str | None = None,
 fiscal_year: int | None = None,
 item: str | None = None, k: int = 5) -> str:
 """Busca fragmentos de texto relevantes en los 10-K del corpus.
 Devuelve k fragmentos, cada uno con su chunk_id para poder citarlo."""

@tool
def read_section(ticker: str, fiscal_year: int, item: str) -> str:
 """Devuelve el TEXTO COMPLETO de una sección. Es CARA: puede devolver
 decenas de miles de tokens."""
```
```

El esquema de respuesta

Vuestro agente devuelve esto, no prosa suelta. Es lo que hace que la evaluación sea automática y lo que permite ejecutar el hold-out en 20 minutos. Podéis añadir campos. No quitéis ni renombréis los que hay.

```
```python
from typing import Literal
```

```

from pydantic import BaseModel, Field

class RespuestaFinanciera(BaseModel):
 """Respuesta trazable a una pregunta sobre informes 10-K."""
 respuesta: str = Field(description="Respuesta en prosa, breve y directa")
 cifra: float | None = Field(default=None, description="Valor numérico, si la pregunta pide uno")
 unidad: str | None = Field(default=None, description="USD, shares, porcentaje...")
 ticker: str | None = None
 ejercicio: int | None = None
 fuente: Literal["xbrl", "texto", "ambas", "ninguna"] = Field(
 description="De dónde sale el dato. 'ninguna' si no está en el corpus"
)
 cita: str | None = Field(default=None, description="Texto literal del informe que respalda la respuesta")
 chunk_id: str | None = Field(default=None, description="Identificador del fragmento citado, para verificar")
 ...

```

### El esquema de pregunta:

```

```json
{
  "id": "g3-007",
  "pregunta": "¿Cuál fue el revenue de NVIDIA en el ejercicio 2024?",
  "familia": "numerica",
  "ticker": "NVDA",
  "fiscal_year": 2024,
  "respuesta_esperada": "60.922 millones de dólares",
  "cifra_esperada": 60922000000.0,
  "unidad": "USD",
  "concept_xbrl": "Revenues",
  "item_esperado": null,
  "ancla_texto": null,
  "ancla_inicio": null,
  "ancla_fin": null,
  "chunk_id_esperado": null,

```

```
"herramienta_esperada": ["get_xbrl_fact"],  
  "autor": "grupo-3"  
}  
...
```